GENIE FOR EXTERNAL USERS · OAUTH CUSTOM IDENTITY CLAIMS

Choosing an Enforcement Model

Community reference, not official Databricks documentation. OAuth custom identity claims are announced on the Databricks *What's coming?* page; check your workspace with `sql/check_availability.sql`.

This guide compares three ways to enforce per-user access when a Genie Agent serves your application's users through OAuth custom identity claims: claim-keyed dynamic views, Unity Catalog ABAC row filter policies, and the two layered together. It explains how each one fails, what each costs, and which to choose. Every number in it comes from a live run against the synthetic examples in this repo.

The short answer

Neither views nor ABAC policies are fail-closed on their own, and they fail in opposite places.

- **Claim-keyed views** stay closed when someone misconfigures them: the application's service principal can read only the views you created, so anything you forgot to expose does not exist for it. They can leak at query time, though. A user who can shape the SQL, which is what Genie does with every question, can learn facts about other users' rows through errors.
- **ABAC row filter policies** block that kind of probe. They fail open when someone misconfigures them: a table or column the policy's tags don't match, or a principal the policy doesn't name, gets the whole table back unfiltered.
- **Layered** keeps both. The views decide what the application can reach. ABAC policies on the base tables decide which rows each user gets, before any condition in the user's query runs. The views keep their own claim check. A single mistake then leaves every user with only their own rows.

  Views decide what the app can reach. Policies decide which rows each user gets. Keep both, so a single mistake never opens the data.

The repo deploys the layered model by default (`--var enforcement=layered`). `--var enforcement=views` deploys the claim-keyed views alone.

Two ways to fail

Configuration mistakes: ABAC fails open

An ABAC policy applies only where its tag conditions match. The Unity Catalog documentation says so directly: when a table doesn't have the columns a policy's conditions match, "the policy does not apply and the data is returned unmasked", and "Tagging is a security boundary." (ABAC core concepts). A policy also applies only to the principals in its `TO` clause.

So with ABAC alone, each of these mistakes returns the whole table to the application:

- a new table granted to the application before anyone tags it;
- one tag removed from a key column;
- a policy that names a service principal or group the application no longer uses.

ABAC does fail closed in the other direction: if a policy's function or tag is deleted, or the compute can't enforce policies, queries are denied (policy evaluation). The documented mitigation for unclassified data is a fallback policy for objects without the expected tags (best practices). It closes the first mistake. It doesn't close the second or third.

Query-time probes: views leak through errors

A view's filter is ordinary SQL, and the optimizer is free to evaluate a user's own condition before the view's filter removes a row. If that condition can fail, the error itself carries information. For example:

```
SELECT count(*) FROM northwind_portal.my_accounts
WHERE 1 / (CASE WHEN balance = 878.95 THEN 0 ELSE 1 END) > 0
```

returns `DIVIDE_BY_ZERO` when any row in the base table has that balance, including other users' rows, and a normal count when none does. Repeated questions turn one bit into a value.

Row filters and ABAC policies are evaluated differently. "Both ABAC and table-level row filters and column masks use a SecureView barrier to prevent predicates with side effects from being pushed across the policy boundary. This protects against side-channel data leakage" (ABAC performance). The row filter documentation names the reason: "errors like 'division by zero' could reveal information about values before filtering or masking" (row filters and column masks).

The isolation suite runs probes like this for every example. Each probe uses a value that exists only in rows the persona may not see, and a control value that exists nowhere. A different outcome between the two is a leak.

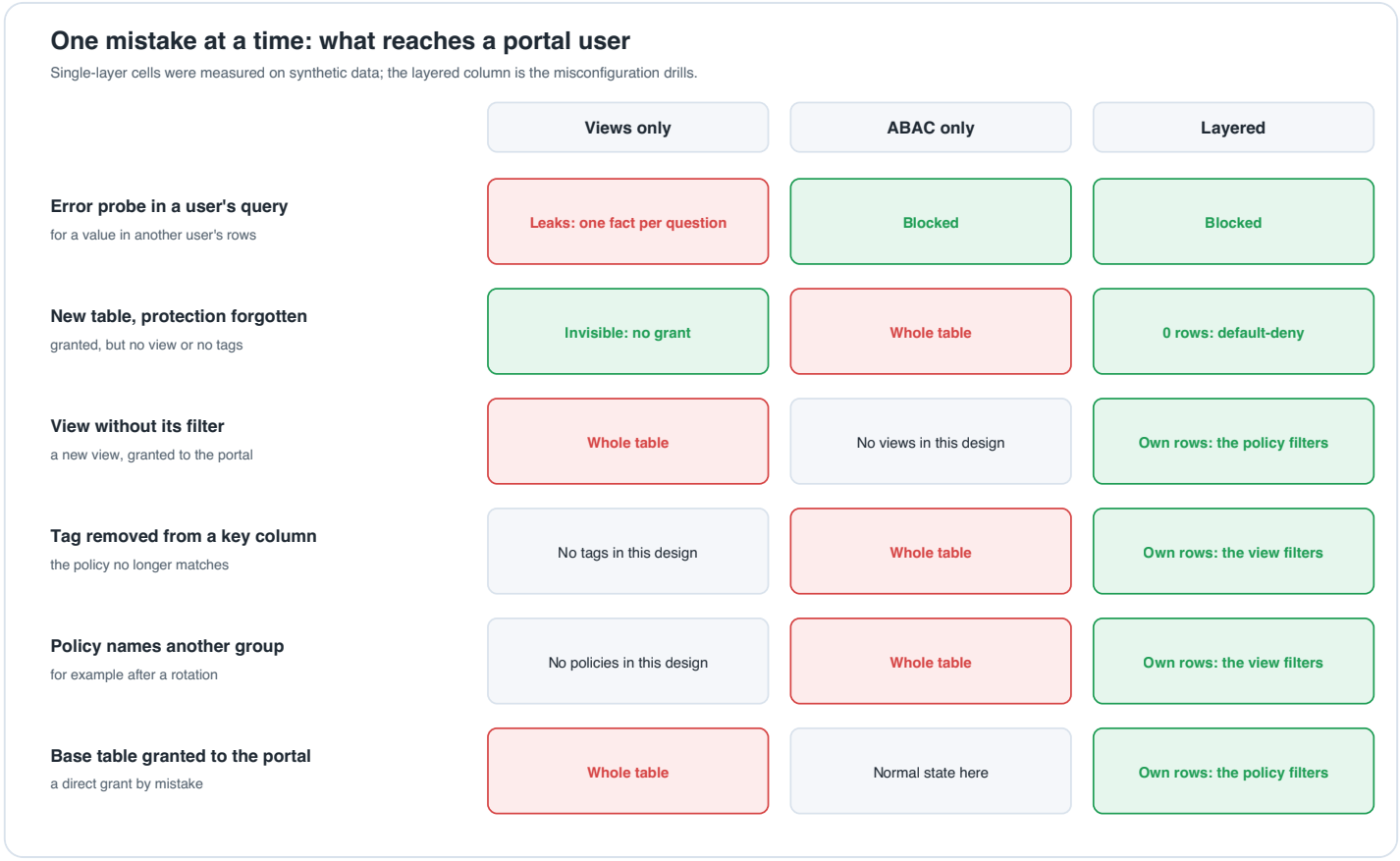
probe	persona	looks for	Views only	Layered
maya-balance-in-my_accounts	maya	another participant's balance	leaked: DIVIDE_BY_ZERO	blocked
raj-participant-in-plan_roster	raj	a Plan B participant, outside Raj's plans	leaked: CAST_INVALID_INPUT	blocked
maya-grant-in-my_access	maya	another user's grant	blocked	blocked
linda-contribution-in-my_contributions	linda	a Plan A participant's contribution; Linda sponsors Plan A but never sees contributions	leaked: DIVIDE_BY_ZERO	blocked

Northwind, live runs on 2026-10-20 (views only) and 2026-10-20 (layered).

probe	persona	looks for	Views only	Layered
ana-spouse-claim-in-my_claims	ana	her spouse's claim; adult household members stay private	leaked: DIVIDE_BY_ZERO	blocked
priya-other-pharmacy-in-pharmacy_remittance	priya	a claim filled at another pharmacy	leaked: CAST_INVALID_INPUT	blocked
theo-cell-in-sponsor_spend_summary	theo	a spend group below Theo's minimum group size of 20	leaked: CAST_INVALID_INPUT	blocked
dana-sponsor-in-sponsor_spend_summary	dana	a sponsor Dana has no grant for; the client list stays private	leaked: DIVIDE_BY_ZERO	blocked

Larkspur, live runs on 2026-10-20 (views only) and 2026-10-20 (layered).

The failure matrix



Single-layer results from live runs on synthetic data; the layered column is the misconfiguration drills below.

The drills make each mistake on a live deployment, measure what the portal returns, and restore the original state. With one layer alone, every drill returned the whole table. With the layers together, each persona still saw only their own rows, and a new untagged table returned nothing.

drill	mistake	layered, with the mistake	one layer alone	result
D1	tag removed from the key column base.account_balances.participant_id	4 rows (expected 4)	444 of 444 rows	held
D2	the row policy names another application's group	4 rows (expected 4)	444 of 444 rows	held
D3	a view without the claim filter is granted to the portal	4 rows (expected 4)	444 of 444 rows	held

drill	mistake	layered, with the mistake	one layer alone	result
D4	a new untagged table is exposed through a view without the claim filter	0 rows (expected 0)	444 of 444 rows	held
D5	the base table base.account_balances is granted to the portal directly	4 rows (expected 4)	444 of 444 rows	held

Northwind, persona maya, live run on 2026-10-20. Every drill restored and verified the original state.

drill	mistake	layered, with the mistake	one layer alone	result
D1	tag removed from the key column base.pharmacy_claims.member_id	27 rows (expected 27)	2079 of 2079 rows	held
D2	the row policy names another application's group	27 rows (expected 27)	2079 of 2079 rows	held
D3	a view without the claim filter is granted to the portal	27 rows (expected 27)	2079 of 2079 rows	held
D4	a new untagged table is exposed through a view without the claim filter	0 rows (expected 0)	2079 of 2079 rows	held
D5	the base table base.pharmacy_claims is granted to the portal directly	27 rows (expected 27)	2079 of 2079 rows	held

Larkspur, persona ana, live run on 2026-10-20. Every drill restored and verified the original state.

Performance

Measured on synthetic data with up to 200 million rows (10,000 tenants, one million members, 1.03 million entitlement rows) on a Small serverless SQL warehouse, with the same functions in every design:

Design	Server time relative to views alone
ABAC alone	0.82x to 1.07x, across 30 query and role combinations at 10 million and 200 million rows
Layered	1.00x to 1.36x, across 8 combinations at 200 million rows: about 0.15 to 0.35 seconds more on queries of about one second

All three returned identical results.

- **Pruning comes from the entitlement lookup.** The claim narrows the lookup to a few entitlement rows, and the optimizer turns that into file pruning on the clustered keys: a scoped user's query read 1 or 2 of 59 files in every design. Cluster protected tables on the columns the policies filter by (tenant, member), and keep the entitlement table small and clustered on the claim column.
- **Layered runs the lookup twice,** once in the view and once in the policy. That second lookup is the overhead in the table.
- **Know the barrier's cost.** A condition that wraps a column in a function, or needs an implicit cast, can't be pushed past a policy, which "can force full table scans... even when the policy UDF resolves to a constant true" (ABAC performance). Genie writes conditions like `year(fill_date) = 2026` often. Pruning from the policy's own lookup still applies, so the risk is concentrated in large tables where users are entitled to most of the data. Benchmark those before you commit.
- **Follow the function guidance in the docs:** simple SQL functions, few arguments, lookup tables small enough to broadcast.
- **Tune with a principal outside the policy.** For the portal's own queries, query history reports 0 bytes read and the query plan stops at the policy.

Operating trade-offs

	Claim-keyed views	ABAC row filter policies
Setup	SQL views and grants	Governed tags, created by an account admin, with ASSIGN limited to a governance group; `MANAGE` on the schema; serverless or Databricks Runtime 16.4+ for the principals the policies apply to
Shape of the data	Any SQL: joins, totals only, minimum group sizes, masks, relationship rules	Row filtering and column masking only
New tables	Invisible until someone writes a view	Covered automatically when tagged; unfiltered when not
Pipelines and administrators	Unaffected: base tables carry no logic	Unaffected when policies name only the portal's group
What the rule reads	Anything	Only tables that carry no policy of their own: Unity Catalog rejects a policy function that reads a protected table
Genie prompt matching	Turn it off on claim-keyed views	Genie excludes row-filtered tables automatically (Genie docs); views over them still need it turned off

	Claim-keyed views	ABAC row filter policies
Reading the rules	One SQL file per view	`SHOW POLICIES`, tags and functions together

---

When to use which

Situation	Choose
Genie or any other path where users shape the SQL, over data where one user's values must stay private	**Layered**
Your application controls every query (fixed reports, parameterized API calls)	Views alone are reasonable
Internal users with their own Databricks identities, many tables, filtering and masking only, strong tag governance	ABAC alone is reasonable
You can't create governed tags yet	Views alone, with the side channel documented and backend error messages kept generic, until you can

Layered costs more to build: row logic lives in the views and in the policy functions, governed tags must exist, and small-group totals must be precomputed (below). When the two places disagree, users lose access rather than gain it, and the isolation suite catches the difference on the next run.

Five rules

1. **The portal reads views only.** Grant the application's group `SELECT` on portal views and nothing on base or governance tables.
2. **Every base table those views read carries a row policy** for the application's group, never for a service principal id, plus a default-deny policy for tables without a grain tag.
3. **Views keep their claim and service principal checks**, so a mistake in tags or policies still leaves users with only their own rows.
4. **Precompute small-group totals.** A `HAVING` clause in a view is not a privacy control (below).
5. **Lock tag assignment to a governance group, and drill each mistake** with the isolation suite and the drills job on every change.

Two subtle cases

Aggregate-only roles. A benefits administrator may see spend by quarter and drug class, but never member rows, and groups with fewer members than the role's minimum stay hidden. If the view computes those totals from member-level claims, the administrator's reach has to include the member rows, and a probe on the group's own columns can reveal a hidden group. Larkspur therefore precomputes `sponsor_spend_cells`, never stores groups smaller than any role's minimum, and its row policy applies each role's own minimum. The `theo-cell` probe above is that case.

Dimension and grant tables. A view that joins an unprotected sponsors table answers "is this employer a client?" through the same kind of probe (the `dana-sponsor` probe above). Protect tenant lists like any other table. The governance tables that the policy functions read can't carry a policy of their own, because Unity Catalog rejects nested policies. Keep them out of every grant, and derive what the rules need (here, plan membership and households) into them.

Status

The custom identity claims announcement on the *What's coming?* page describes dynamic views (AWS). The layered model adds Unity Catalog ABAC row filter policies, a documented Unity Catalog feature, underneath the claim-keyed views. Run the availability check and the isolation suite in your own workspace before you rely on either.